

Пайплайн газовых тревог

грамотное описание (свой weave / B2)

Содержание

1	Задача	1
2	Доменные типы	1
3	Таблица порогов	1
4	Условие и спецификация тревоги	2
5	Сборка пайплайна	2

Здесь первоисточник — этот самый .ml-файл: он компилируется как есть, его видят тесты, coverage и аудит. Развёрнутая проза живёт в блоках `@doc` ..., а маленький скрипт `weave.py` собирает из них и из кода документ в порядке файла. Никакого шага извлечения кода нет: то, что запускается, и есть единственный источник.

```
open Miniflink
```

1 Задача

Нам приходит поток показаний газа по зонам. Для каждой зоны действует свой порог, и пороги меняются на ходу — их присылает отдельный поток уставок. Нужно поднимать тревогу, когда показание превышает текущий порог своей зоны, и снимать её при возврате в норму.

2 Доменные типы

Три записи. `reading` — сырое показание: зона и концентрация в ppm. `setpoint` — уставка: для какой зоны какой порог. `checked` — показание, уже дополненное действующим порогом; именно его увидит правило тревоги.

```
type reading = { zone : string; ppm : float }
type setpoint = { sp_zone : string; max_ppm : float }
type checked = { c_zone : string; c_ppm : float; c_max : float }
```

3 Таблица порогов

Пороги живут не в коде, а в данных — их присылает поток уставок. `Table.build` превращает поток в таблицу «зона -> уставка», которая обновляется сама по мере прихода новых значений. Ключ — имя зоны.

```
let thresholds_of (setpoints : setpoint Mf_event.t Stream.t)
  : (string, setpoint) Table.t =
  Table.build ~key:(fun s -> s.sp_zone) setpoints
```

4 Условие и спецификация тревоги

Порог теперь поле события, поэтому условие сравнивает два поля одного `checked`: проблема, когда `c_ppm` выше `c_max`; снятие, когда вернулось на уровень порога или ниже. `Trigger.custom` принимает ровно такую пару предикатов.

```
let gas_condition =
  Trigger.custom
    ~problem:(fun c -> c.c_ppm > c.c_max)
    ~recovery:(fun c -> c.c_ppm <= c.c_max)
```

Спецификация добавляет к условию тексты тревоги и снятия. Триггер сам ведёт автомат на каждую зону: одна активная тревога на зону, снятие парно тревоге.

```
let gas_spec =
  Trigger.create ~name:"gas" ~condition:gas_condition
    ~produce_alert:(fun ~key ~value ~ts:_ ->
      Printf.sprintf "GAS %s: %.2f > %.2f" key value.c_ppm value.c_max)
    ~produce_recovery:(fun ~key ~ts:_ -> Printf.sprintf "CLEAR %s" key)
    ()
```

5 Сборка пайплайна

Связываем всё в цепочку. Показания сперва оборачиваются в `checked` с заведомо безопасным порогом `infinity` (пока настоящий не подмешан — тревоги не будет). Затем `enrich` подставляет действующий порог из таблицы. Затем событие превращается в пару «(зона, checked)», как ждёт триггер. И наконец `Trigger.of_stream` выдаёт поток тревог.

```
let gas_alerts ~(setpoints : setpoint Mf_event.t Stream.t)
  (readings : reading Mf_event.t Stream.t) : string Mf_event.t Stream.t =
  let table = thresholds_of setpoints in
  readings
  |> Stream.map (Mf_event.map_value (fun r ->
    { c_zone = r.zone; c_ppm = r.ppm; c_max = infinity }))
  |> Pipe.enrich (module struct
    type t = checked
    let key c = c.c_zone
  end)
    ~from:table
    ~merge:(fun c sp -> match sp with
      | Some s -> { c with c_max = s.max_ppm }
      | None -> c)
  |> Stream.map (Mf_event.map_value (fun c -> (c.c_zone, c)))
  |> Trigger.of_stream gas_spec
```